Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251176**
Nama Lengkap	**David Alexsi Cahyadi**
Minggu ke / Materi	**05 / Modular Programming**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Fungsi, Argument dan Parameter

Pada bahasa pemrograman Python, fungsi merupakan sekumpulan perintah yang digabungkan menjadi satu kesatuan untuk menjalankan tugas tertentu dan dapat digunakan kembali sesuai kebutuhan. Fungsi berperan penting dalam konsep *modular programming* karena memungkinkan program yang besar dipecah menjadi bagian-bagian kecil yang lebih terstruktur dan mudah dipahami. Dalam Pyhton terdapat dua jenis fungsi, yaitu fungsi bawaan (built-in function) seperti **input()** dan **print()**, serta fungsi yang dibuat sendiri oleh programmer.

Pendefinisian fungsi dilakukan dengan menggunakan keyword **def** yang diikuti dengan nama fungsi dan parameter. Parameter merupakan variabel yang digunakan untuk menerima nilai dan masukan dari luar fungsi. Nilai yang dikirimkan ke dalam fungsi saat pemanggilan disebut sebagai argument. Ketika fungsi dipanggil, nilai argument akan menggantikan parameter yang terdapat pada definisi fungsi sesuai dengan urutannya, kemudian perintah di dalam fungsi akan dijalankan hingga menghasilkan suatu proses tertentu. Dengan adanya fungsi, kode program dapat ditulis secara lebih efisien karena dapat digunakan berulang kali tanpa perlu menuliskan ulang perintah yang sama.

```python
# definisikan fungsi tambah terlebih dahulu
def tambah(a, b):
    hasil = a + b
    return hasil

# panggil fungsi tambah dengan dua arguments berupa nilai: 10 dan 5
c = tambah(10, 5)
print(c)
```

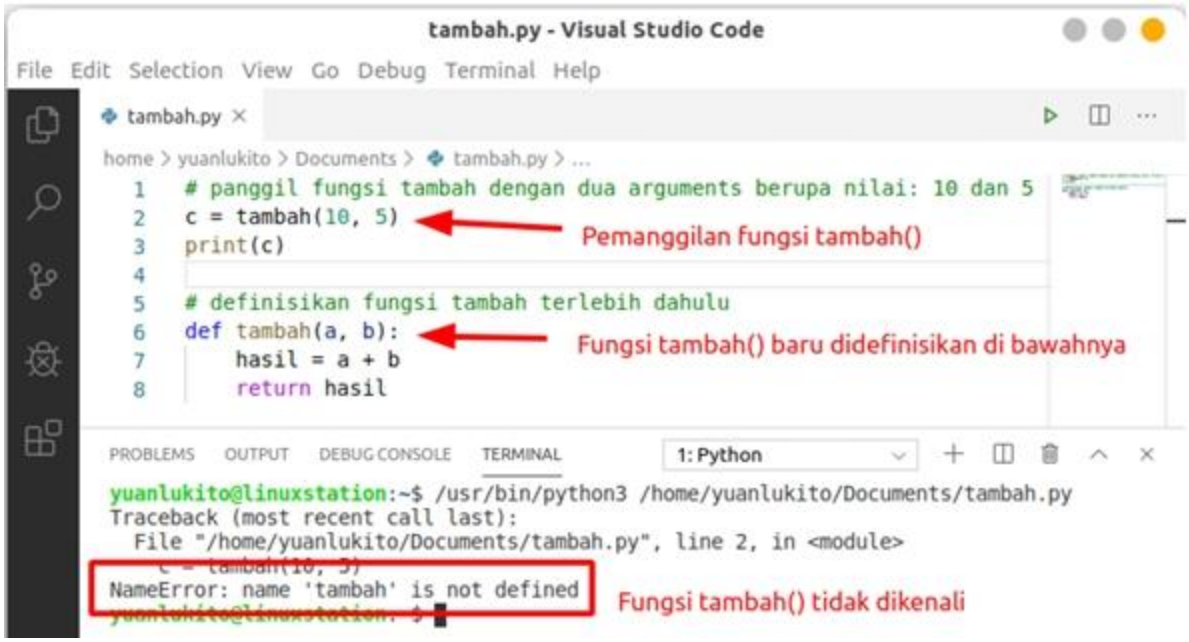
Gambar 5.1: Contoh penggunaan def, Sumber: https://shorturl.at/iz01a

```
yuanlukito@linuxstation: ~/Documents
yuanlukito@linuxstation:~/Documents$ python3 tambah.py
15
```

Gambar 5.2: Hasil dari pemanggilan fungsi tambah(), Sumber: https://shorturl.at/iz01a

Return Value

Berdasarkan hasil yang dikeluarkan, fungsi dalam Python dapat dibedakan menjadi dua jenis, yaitu fungsi yang tidak mengembalikan nilai dan fungsi yang mengembalikan nilai. Fungsi yang tidak mengembalikan nilai biasanya hanya menjalankan suatu perintah tertentu seperti menampilkan output ke layar dan dikenal sebagai *void function*. Fungsi jenis ini sebenarnya tetap mengembalikan suatu nilai khusus yaitu **None**, meskipun tidak dapat digunakan untuk proses selanjutnya.



Gambar 5.3: Fungsi tambah() belum dikenali karena didefinisikan di bawahnya,

Sumber: https://shorturl.at/iz01a

Berbeda dengan fungsi yang mengembalikan nilai, fungsi jenis ini menggunakan keyword **return** untuk mengembalikan hasil dari proses yang dilakukan di dalam fungsi. Penggunaan **return** berfungsi untuk mengeluarkan nilai hasil dari proses sekaligus mengakhiri eksekusi fungsi tersebut. Nilai yang dikembalikan dapat disimpan ke dalam variabel lain sehingga dapat digunakan kembali dalam proses lanjutan, misalnya untuk melakukan perhitungan rata-rata atau operasi matematika lainnya.

Optional Argument dan Named Argument

Pada Python, fungsi dapat memiliki parameter yang bersifat opsional atau disebut sebagai *optional argument*. Optional argument adalah parameter yang telah dimiliki nilai bawaan (*default value*) yang didefinisikan pada saat pembuatan fungsi. Dengan adanya nilai default tersebut, parameter tidak wajib diisi ketika fungsi dipanggil. Jika pada saat pemanggilan

fungsi tidak diberikan nilai untuk parameter opsional, maka secara otomatis fungsi akan menggunakan nilai default yang telah dilakukan sebelumnya.

```
def hitung_belanja(belanja, diskon=0):
    bayar = belanja - (belanja * diskon)/100
    return bayar
```

Gambar 5.4: Penggunaan parameter opsional pada fungsi, Sumber: https://shorturl.at/iz01a

Penggunaan optional argument memberikan fleksibilitas dalam pemanggilan fungsi karena pengguna dapat memilih untuk memberikan nilai tambahan atau tidak, tergantung kebutuhan. Hal ini sangat berguna dalam situasi tertentu, misalnya pada fungsi perhitungan diskon atau perhitungan tagihan, di mana suatu parameter mungkin tidak selalu digunakan. Dengan adanya nilai default, fungsi tetap dapat dijalankan tanpa menimbulkan kesalahan meskipun parameter opsional tidak diisi.

Selain opsional argument, Python juga mendukung penggunaan *named argument*. Named argument adalah cara pemanggilan fungsi dengan menyebutkan nama parameter secara langsung saat mengirimkan nilai argument. Dengan metode ini, urutan argument tidak harus sama dengan urutan parameter pada definisi fungsi. Hal ini meningkatkan keterbacaan kode serta meminimalkan kesalahan dalam pengiriman nilai, terutama jika jumlah parameter cukup banyak. Penggunaan named argument membuat program menjadi lebih jelas karena setiap nilai yang diberikan langsung terlihat pasangan parameternya.

```
def hitung_belanja(belanja, diskon=0):
    bayar = belanja - (belanja * diskon)/100
    return bayar

print(hitung_belanja(100000))
print(hitung_belanja(100000, 10))
print(hitung_belanja(100000, 50))
```

Gambar 5.5: Contoh memanggil fungsi, Sumber: https://shorturl.at/iz01a

Anonymous Function (Lambda)

Anonymous function atau yang dikenal sebagai *lambda function* merupakan fungsi tanpa nama yang digunakan untuk membuat operasi sederhana secara ringkas. Berbeda dengan fungsi biasa yang didefinisikan menggunakan keyword **def**, lambda function menggunakan keyword **lambda** dan dituliskan dalam satu baris ekspresi. Fungsi ini biasanya digunakan ketika diperlukan fungsi sederhana yang tidak terlalu kompleks dan tidak membutuhkan banyak baris perintah.

```
def kelipatan_sembilan(angka):
    if angka % 9 == 0:
        return True
    else
        return False
```

Gambar 5.6: Contoh bentuk fungsi, Sumber: https://shorturl.at/iz01a

```
kelipatan_sembilan = lambda angka: angka % 9 == 0
```

Gambar 5.7: Contoh bentuk penggunaan lambda, Sumber: https://shorturl.at/iz01a

Struktur lambda function terdiri dari tiga bagian utama, yaitu keyword **lambda**, parameter (atau yang dapat disebut bound variable), dan ekspresi yang akan diproses. Ekspresi tersebut langsung menghasilkan nilai sebagai output tanpa perlu menggunakan keyword **return**, karena secara otomatis nilai hasil ekspresi akan dikembalikan. Hal ini membuat lambda function lebih praktis dibandingkan fungsi biasa.

Lambda function sering digunakan dalam situasi seperti pemrosesan data atau operasi sederhana yang hanya membutuhkan satu ekspresi, misalnya pengecekan kondisi atau perhitungan matematika singkat. Namun demikian, lambda function memiliki keterbatasan karena hanya dapat memuat satu ekspresi dan tidak dapat digunakan untuk proses yang kompleks atau membutuhkan banyak langkah. Oleh karena itu, lambda function lebih cocok digunakan untuk penyederhanaan fungsi yang sederhana dan spesifik.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Github:https://github.com/DavidCahyadi1013/71251176_David-Alexsi-Cahyadi.git

Latihan 5.1

Soal:

Latihan 5.1 Buatlah sebuah fungsi yang dapat menentukan apakah ketiga parameter memenuhi

semua ketentuan berikut ini:

• Ketiga parameter tersebut nilainya berbeda semua.

• Ada kemungkinan jika diambil dua parameter dan dijumlahkan hasilnya sama dengan

parameter lainnya (yang tersisa).

Fungsi tersebut akan menghasilkan nilai True jika semua ketentuan tersebut dipenuhi. Jika tidak terpenuhi maka fungsi akan menghasilkan nilai False. Fungsi anda harus diberi nama cek_angka().

Source Code:

```
#Latihan 5.1
def cek_angka(a,b,c):
 berbeda = (a != b) and (a != c) and (b != c)
 sama = (a +b == c) or (a + c == b) or (b + c == a )
 return berbeda and sama
```

Contoh print:

```
print(cek_angka(2,3,5))
print(cek_angka(4,4,8))
print(cek_angka(1,2,4))
```

```
print(cek_angka(5,2,3))
```

Output:

```
True
False
False
True
```

Penjelasan:

Fungsi **cek_angka(a,b,c)** dibuat untuk menentukan apakah tiga parameter yang diberikan memenuhi dua ketentuan tertentu. Ketentuan pertama adalah ketiga parameter harus memiliki nilai yang berbeda satu sama lain. Ketentuan kedua adalah harus terdapat kemungkinan bahwa jumlah dua dari ketiga parameter tersebut sama dengan parameter yang tersisa.

Fungsi akan mengecek apakah **a**, **b**, dan **c** saling berbeda menggunakan operator perbandingan (**!=**). Kemudian, fungsi memeriksa apakah ada kombinasi dua angka yang jumlahnya sama dengan angka yang tersisa, yaitu dengan mengecek kondisi **a + b == c**, **a + c == b**, atau **b + c == a**. Hasil akhir dari fungsi operasi logika **and** dari kedua pemeriksaan tersebut, sehingga fungsi hanya mengembalikan nilai **True** jika kedua syarat terpenuhi, dan **False** jika salah satunya tidak terpenuhi.

Latihan 5.2

Soal:

Latihan 5.2 Buatlah sebuah fungsi yang dapat menentukan apakah minimal dua dari tiga

parameter yang diberikan memiliki digit paling kanan yang sama. Fungsi tersebut menghasilkan

nilai True jika memenuhi dan False jika tidak memenuhi. Gunakan fungsi tersebut untuk

mengecek beberapa test-case berikut ini:

• Input = 30, 20, 18. Output yang diharapkan = True

• Input = 145, 5, 100. Output yang diharapkan = True

• Input = 71, 187, 18. Output yang diharapkan = False

• Input = 1024, 14, 94. Output yang diharapkan = True

• Input = 53, 8900, 658. Output yang diharapkan = False

Ketiga bilangan tersebut diinputkan oleh pengguna, sehingga anda perlu membaca input dari

pengguna. Fungsi anda harus diberi nama cek_digit_belakang().

Source Code:

```
#Latihan 5.2
def cek_digit_belakang(a,b,c):
 digita = a % 10
 digitb = b % 10
 digitc = c % 10
 if digita == digitb or digita == digitc or digitb == digitc:
  return True
 else:
  return False

a1 = int(input("Masukkan bilangan pertama: "))
a2 = int(input("Masukkan bilangan kedua: "))
a3 = int(input("Masukkan bilangan ketiga: "))

hasil = cek_digit_belakang(a1,a2,a3)
print ("Hasil:", hasil)
```

Output:

```
Masukkan bilangan pertama: 10002
Masukkan bilangan kedua: 2432
Masukkan bilangan ketiga: 9832
Hasil: True
```

Penjelasan:

Program dimulai dengan mendefinisikan fungsi **cek_digit_belakang(a, b, c)** yang bertujuan untuk mengecek apakah minimal dua dari tiga bilangan memiliki digit kanan yang sama. Di dalam fungsi, setiap bilangan diambil digit terakhirnya menggunakan operator modulo 10 (**% 10**). Hasilnya disimpan dalam variabel **digita**, **digitb**, **digitc**.

Setelah itu, dilakukan pengecekan menggunakan percabangan **if** untuk membandingkan ketiga digit tersebut. Jika terdapat minimal satu pasang digit yang sama (misalnya **digita == digitb** atau pasangan lainnya), maka fungsi akan mengembalikan nilai **True**. Jika tidak ada digit yang sama, fungsi akan mengembalikan **False**.

Pengguna diminta memasukkan tiga bilangan kemudian dikonversi ke tipe bilangan bulat (integer). Ketiga nilai tersebut dikirim ke fungsi, lalu hasilnya disimpan dan ditampilkan ke layar. Contoh terdapat input 10002, 2432, dan 9832, ketiganya memiliki digit terakhir 2, sehingga fungsi menghasilkan **True.**

Latihan 5.3

Soal:

Latihan 5.3 Buatlah fungsi-fungsi konversi suhu menggunakan lambda function. Fungsi-fungsi

yang harus anda implementasikan:

• Celcius to Fahrenheit. F = (9/5) ∗C +32

• Celcius to Reamur. R = 0.8 ∗C

Berikan contoh penggunaannya untuk test-case berikut ini:

• Input C = 100. Output F = 212.

• Input C = 80. Output R = 64.

• Input = 0. Output F = 32.

Source Code:

```
#Latihan 5.3
f = lambda c: (9/5) * c + 32
r = lambda c: 0.8 * c

print(" c = 100 ->  F =", f(100))
print(" c = 80 ->  R =", r(80))
print(" c = 0 ->  F =", f(0))
```

Output:

```
c = 100 ->  F = 212.0
c = 80 ->  R = 64.0
c = 0 ->  F = 32.0
```

Penjelasan:

Program dimulai dengan mendefinisikan dua buah fungsi menggunakan *lambda function*, yaitu fungsi **f** dan fungsi **r**. Fungsi **f** digunakan untuk mengonversi suhu dari Celcius ke Fahrenheit dengan rumus **F = (9/5) * C + 32**. Sementara itu, fungsi **r** digunakan mengonversi suhu dari Celcius ke Reamur dengan rumus **R = 0.8 * C**. Karena menggunakan kata kunci **def**, namun tetap menerima satu parameter yaitu **c** sebagai nilai suhu dalam Celcius.

Setelah fungsi didefinisikan, program kemudian memanggil fungsi tersebut pada bagian **print()** untuk menampilkan hasil konversi sesuai dengan test-case yang diberikan pada soal. Fungsi **f(100)** dijalankan untuk menghitung konversi 100°C ke Fahrenheit sehingga menghasilkan 212.0. Kemudian, fungsi **r(80)** dijalankan untuk menghitung konversi 80°C ke Reamur dan menghasilkan 64.0. Lalu, fungsi **f(0)** digunakan untuk mengonversi 0°C ke Farenheit dan menghasilkan 32.0.

Output yang ditampilkan sesuai dengan perhitungan berdasarkan rumus konversi suhu yang telah ditentukan. Angka hasil konversi ditampilkan dalam bentuk desimal (floating point) karena pada perhitungan digunakan operasi pembagian dan perkalian bilangan pecahan.